Homework Assignment 1

The Art of Analyzing Big Data: The Data Scientist's Toolbox

Tal Klein: 209234103

Link to tasks application - CLICK HERE

Task 1: Baby Names Explorer

What I Built and How It Works

I built an interactive Streamlit dashboard that lets users explore United States baby naming trends spanning from 1910 to 2014, using the Social Security Administration StateNames dataset of approximately 5.6 million records. On first launch the app auto-initialises a SQLite database from the CSV file using pandas to_sql for fast bulk insertion, then creates indexes and caches key metadata (total row count, year range) in a small app_metadata table so that subsequent startups are instant without scanning the full table.

The schema is a single table, national_names, with columns Id (PRIMARY KEY), Name, Year, Gender, State, and Count. Six composite indexes cover the most common access patterns: (Name, Year) for popularity-over-time queries; (Year, Gender) for gender-split aggregations; (State) and (Name, State, Year) for regional comparisons; and two additional covering indexes for performance on 5.6 million rows.

The UI is organised into five tabs. The Time Series tab lets users type comma-separated names and instantly see a Plotly line chart with a toggle between raw birth counts and percentage of all births that year, making it easy to compare popular names across generations with different total birth volumes. The Query Builder tab provides a free-form SQL panel with three pre-built example queries and blocks any non-SELECT statement with a user-friendly message. The Diversity Trends tab shows a dual-axis chart plotting unique names per year on the left axis and average births per name on the right, visualising the diversity explosion since the 1950s. The Pattern Discovery tab surfaces three pre-computed findings with the underlying SQL shown in expanders: (1) the naming diversity explosion from roughly 1,693 unique names in 1910 to 9,585 in 2014; (2) the Arya Effect where the name Arya spiked from 249 births in 2010 to 1,574 in 2014 immediately after Game of Thrones premiered; and (3) regional divergence where Jose dominates in California and Texas but barely appears in Montana. The Schema tab shows table structure, index definitions, and row counts.

Technologies and Libraries: Choices and Rationale

I chose direct sqlite3 with no ORM because this is a read-only analytics workload. Raw SQL with parameterised queries is simpler, faster, and more transparent than ORM abstractions when all queries are SELECT statements and there are no object relationships to navigate. Pandas was used exclusively for the initial CSV ingestion via to_sql, which inserts millions of rows in a single transaction and is orders of magnitude faster than looping with individual

INSERTs. Plotly Express and Graph Objects provide interactive charts with zoom, hover tooltips, spike lines, and dual-axis support, which is essential for a dataset spanning over 100 years. I chose Plotly over matplotlib because Streamlit renders Plotly charts natively as interactive HTML components. Streamlit's @st.cache_resource decorator caches the database connection across reruns, and @st.cache_data caches computed query results such as the diversity statistics so they are not re-executed on every interaction.

Challenges Encountered and How I Solved Them

Performance on 5.6 million rows was the primary challenge. A naive implementation with no indexes made even simple queries take several seconds. I solved this by profiling the access patterns and creating six targeted composite indexes, reducing query time to under 100 ms for all common operations. I also introduced the app_metadata table to store the total row count and year boundaries at insert time, so the schema tab loads instantly without a full COUNT(*) scan on every startup.

Task 2: Oscar Actor Explorer

What I Built and How It Works

I built a Streamlit app that normalises a flat Oscar nominations CSV into a four-table relational database using SQLAlchemy ORM, then enriches actor profiles with live data fetched from the Wikipedia REST API. No raw SQL string is used anywhere in the application logic; all queries are expressed through SQLAlchemy's session API.

The schema follows a classic fact/dimension model. Person stores unique names, Film stores unique titles, and Category stores Oscar category names. Nomination is the fact table, linking a person, film, and category to a specific year and ceremony number, with a boolean winner field and a unique constraint preventing duplicate rows. SQLAlchemy relationship() and back_populates directives make navigating from a Person to all their nominations, or from a Nomination back to its Person and Film, straightforward without manual JOIN queries.

The CSV preprocessing pipeline is non-trivial. It splits multi-person credits, for example "Brad Pitt/Dede Gardner/Jeremy Kleiner" on a single row, into separate Nomination rows, one per person. A canon_category field groups historically renamed Oscar categories so that queries spanning multiple decades return consistent results. After preprocessing, dimension tables are populated first with unique entities, then nomination rows are inserted with foreign key references, all via SQLAlchemy bulk_insert_mappings for performance.

The app features an actor and director profile page where the user selects a name from a dropdown. The profile combines database data (nominations, wins, categories, years active, film list) with Wikipedia enrichment (biography summary, birth date, and photo via three sequential REST API calls). Computed insights include win rate, years between first nomination and first win, and percentile comparisons to other nominees in the same category. Three analytical discovery sections highlight the longest wait-to-win timelines computed using correlated subqueries, multi-category versatility for people nominated as both actor and director, and actor statistics ranked by total nominations or wins.

Technologies and Libraries: Choices and Rationale

SQLAlchemy ORM was chosen over direct sqlite3 because the analytical queries are inherently relational. Computing the wait-to-win metric requires two correlated subqueries (first nomination year per person and first win year per person) joined on person name, with the difference computed in the SELECT clause. Expressing this in SQLAlchemy's composable API is safer and more maintainable than constructing and maintaining long SQL strings. SQLAlchemy also provides joinedload() for eager loading related objects in a single query, avoiding N+1 query problems when building profile pages. I chose SQLAlchemy over lighter alternatives such as PonyORM or Peewee because it is the industry standard with the best documentation, broadest feature set, and strongest support for advanced constructs such as subqueries, case expressions, and ORM-level column aliasing. The Requests library was used for Wikipedia API calls because it is the standard Python HTTP client, and the Wikipedia REST API is straightforward to consume without a specialised SDK.

Challenges Encountered and How I Solved Them

Bulk loading through the ORM was initially too slow. SQLAlchemy's standard session.add() loop generates one INSERT per row. For a dataset with tens of thousands of nominations this was unacceptably slow. The fix was to switch to bulk_insert_mappings() with batches of 15,000 rows, which delegates to a multi-row INSERT statement and reduced load time from minutes to seconds.

Wikipedia enrichment required three separate API calls per actor (search, summary, and image endpoint) with fallback logic at each step: if the search returns no results, or the summary lacks a photo, the app gracefully degrades to showing only the available data. A data versioning mechanism was also implemented; a data_version value stored in the app_metadata table is checked at startup, and if it does not match the expected version, the entire database is dropped and rebuilt to prevent stale or partially-migrated data from causing silent errors.

Task 3: Pokemon Battle Arena

What I Built and How It Works

I built a full turn-based Pokemon battle simulator in Streamlit, entirely backed by SQLite. All Pokemon stats, type effectiveness multipliers, and battle history are stored in the database; nothing is hardcoded at runtime except the initial data definitions used to populate the DB on first launch. The app has four tabs: Battle Arena, Analysis, Schema and Data, and Battle Mechanics.

The database has four tables. pokemon holds stats for approximately 800 Pokemon including id, name, type1, type2, hp, attack, defense, sp_atk, sp_def, speed, generation, legendary, and total. pokemon_original is an exact copy created at initialisation time and never modified; it serves as the restore point for the cheat system. type_effectiveness stores the full 18x18 type chart as 324 rows (attacking_type, defending_type, multiplier), enabling SQL lookups for any type matchup. battle_history logs the outcome of every fight with team names, winner, cheats used, and a timestamp.

The battle flow starts with team selection where the player chooses one to three Pokemon from a searchable list with a stat preview card showing HP, Attack, Defense, Sp. Atk, Sp. Def, and Speed as normalised bars. The AI then picks a random team of the same size. Each turn the engine sorts all alive Pokemon by Speed, with ties broken randomly, and processes attacks in that order. Damage uses a simplified Pokemon Generation V formula: base equals ((2 times 50 divided by 5 plus 2) times 60 times (A divided by D)) divided by 50 plus 2, then multiplied by type effectiveness queried from the DB and a random factor between 0.85 and 1.0. The attacker uses whichever of Attack or Sp. Atk is higher; the defender uses the corresponding defensive stat. Dual-type defenders get both type multipliers applied sequentially. The battle ends when all Pokemon on one side faint.

Four cheat codes each perform a real SQL write operation: UPUPDOWNDOWN executes UPDATE pokemon SET hp = hp * 2; GODMODE sets defense and sp_def to 999; STEAL INSERTs a copy of the opponent's strongest Pokemon with a "_stolen" name suffix; NERF halves the opponent's attack, sp_atk, and speed. A cheat audit tab runs SELECT queries comparing the live pokemon table against pokemon_original to detect modified stats and stolen entries. Before each new battle the DB is restored via DELETE FROM pokemon followed by INSERT INTO pokemon SELECT * FROM pokemon_original. The Analysis tab shows two data insights: Dragon-type combinations have the highest average total stats, and there is mild power creep across generations visible in a dual-axis Plotly chart.

Technologies and Libraries: Choices and Rationale

Direct sqlite3 was chosen over SQLAlchemy because the game's database operations are simple CRUD statements. The cheat codes in particular are most naturally expressed as literal UPDATE and INSERT SQL strings since the intent is to demonstrate SQL write operations explicitly; an ORM layer would obscure that educational aspect. WAL (Write-Ahead Logging) journal mode was enabled on every connection to allow concurrent reads during write operations, which is essential on Streamlit Cloud where multiple threads render tabs simultaneously. Streamlit's session_state manages the entire game state machine (select, battle, done), team compositions, current HP values, the battle log list, and the list of

cheats applied, all of which must survive across reruns triggered by button clicks. Plotly was used for the power-creep dual-axis chart and the type-effectiveness heatmap with a custom colourscale (red for 0, white for 1, green for 2).

Challenges Encountered and How I Solved Them

Thread safety on Streamlit Cloud was the most significant challenge. The original implementation decorated get_conn() with @st.cache_resource, returning a single shared sqlite3.Connection object. Streamlit Cloud renders all tabs concurrently in separate threads. When one thread called conn.close() on the cached connection, it was permanently invalidated; the next thread received the same closed object and raised sqlite3.OperationalError. The fix was to remove @st.cache_resource from get_conn() entirely so that each call creates a fresh connection with timeout=30 and journal_mode=WAL. All database functions were updated to use try/finally blocks that close the connection after use, preventing file-handle leaks.

A CSS rendering bug caused battle card styles to appear as literal visible text during battles. When the glow_css variable was an empty string for non-active Pokemon, the multi-line HTML style attribute contained a blank line. The CommonMark specification used by Streamlit's Markdown renderer ends an HTML block on a blank line, after which content is parsed as Markdown. Lines with eight spaces of indentation following the blank line were rendered as Markdown code blocks, displaying raw CSS text such as "transition: box-shadow 0.3s, transform 0.15s;" on screen. The fix was to move glow_css inline on the same line as the transition property, ensuring no blank line is ever produced regardless of the variable's value.

Task 4: SQL Learning Game (Minesweeper)

What I Built and How It Works

I built a classic Minesweeper game where hitting a mine does not end the game; instead it triggers an SQL rescue challenge in a modal dialog. The player defuses the mine by correctly answering a SQL question about a real NBA database. SQL difficulty escalates with each mine hit, naturally teaching progressively harder concepts across five levels. The game has three board sizes: Easy (9x9 grid, 10 mines), Medium (16x16, 40 mines), and Expert (16x30, 99 mines). A persistent leaderboard tracks scores, and a scoring formula rewards speed, penalises mine hits and hint usage, and scales with difficulty.

The NBA database has four tables: teams (30 teams with win/loss records, conference, division), players (150+ players with per-game stats: points, rebounds, assists, steals, blocks, and shooting percentages), games (season results linking home and away teams with scores and dates), and ms_leaderboard (tracking player nicknames, difficulty, completion time, mines hit, max SQL level reached, hints used, and calculated score). Data is fetched live from the nba_api Python library on first run, with a hardcoded fallback of 100 rows covering the 2024-25 season if the API is unavailable or rate-limited.

The five SQL levels are tied to cumulative mine hits. Level 1 covers SELECT * for basic retrieval. Level 2 introduces WHERE, AND, and OR for filtering. Level 3 teaches ORDER BY and LIMIT for sorting and ranking. Level 4 covers GROUP BY with aggregate functions COUNT, AVG, SUM, and MAX. Level 5 requires JOIN queries combining teams with players or games tables. Each level has six or more randomised question generator functions that pick a random team or player and compute the correct answer from the live database, so every playthrough presents different questions. When a player answers incorrectly, contextual feedback shows the error message or row count of their query. Answer validation uses fuzzy matching so that "Celtics" matches "Boston Celtics", "32" matches 32.0, and full DataFrame comparison is used for query-match challenges.

The Minesweeper board is rendered using a JavaScript iframe component. Click events are intercepted by JavaScript and written as a coordinate string to a hidden Streamlit text_input, which Streamlit reads on the next rerun. Blank cell revelation uses a BFS flood-fill algorithm with a collections.deque to auto-reveal all adjacent empty cells in a single click. The @st.fragment decorator isolates board rendering so that individual cell clicks trigger a local re-render rather than a full page reload, keeping the game responsive.

Technologies and Libraries: Choices and Rationale

Streamlit's @st.dialog decorator creates a modal overlay for the SQL rescue challenge. The modal prevents the player from interacting with the board while a question is active, enforcing the game rule that a mine must be defused before play can continue. @st.fragment was chosen for board rendering because Minesweeper requires high-frequency click handling; without fragments, every cell click would re-run the entire page including the SQL question generator, the leaderboard query, and all other tabs, which would be noticeably slow. The nba_api library provides an official Python client for NBA statistics, giving access to player per-game stats, team standings, and game logs without scraping. The NBA dataset makes SQL queries concrete and engaging because "Who is the

leading scorer on the Lakers?" is more motivating than querying abstract synthetic data. collections.deque was chosen for the flood-fill because it provides O(1) popleft() for BFS traversal, and Minesweeper boards can have large contiguous empty regions that must be revealed in a single user action.

Challenges Encountered and How I Solved Them

The nba_api library is an external dependency that rate-limits requests and occasionally times out. If database initialisation failed halfway through, for example after fetching teams but before fetching players, subsequent runs would find a partially-populated database and generate incorrect SQL questions. The fix was to wrap the entire API fetch sequence in a try/except block: on any exception the partial database file is deleted and the app falls back to a _create_fallback_data() function which inserts 100 hardcoded rows covering key players and teams from the 2024-25 season. Individual API calls also have a 0.7-second sleep between them to respect rate limits.

Streamlit does not natively support right-click events or complex click handling on custom HTML elements, but Minesweeper requires both left-click to reveal and right-click to flag a cell. The solution was to embed the board as an HTML and JavaScript component via st.components.v1.html(). JavaScript event listeners inside the iframe intercept both click types and write a coordinate and action string such as "5,3,reveal" or "5,3,flag" to a Streamlit text_input element. Streamlit detects the text_input change on the next rerun and processes the action. This creates a clean bridge between JavaScript interaction and Python game logic without requiring any custom Streamlit component or external plugin.

Managing state between the fragment and the dialog was complex. @st.fragment runs in a local scope and triggers partial reruns, but major game events such as a mine hit, game completion, or SQL answer submission need a full application rerun to update all components. The solution was a flag pattern: the fragment sets a session state flag and calls st.rerun(scope="app"), which triggers a full-page rerun that the outer application code detects and acts on. NBA position data from the API was also inconsistent, returning "PG", "Point Guard", None, or hybrid formats like "G-F". A _normalize_position() mapping dictionary with over 30 variant entries was written to convert all formats to a canonical set of five positions, ensuring GROUP BY position queries return clean, consistent results.